

GENETIC PROGRAMING

```
[6] 4s
import copy

def get_all_nodes(node):
    nodes = [node]
    for child in node.children:
        nodes.extend(get_all_nodes(child))
    return nodes

def crossover(parent1, parent2):
    child1, child2 = copy.deepcopy(parent1), copy.deepcopy(parent2)
    nodes1, nodes2 = get_all_nodes(child1), get_all_nodes(child2)

    n1, n2 = random.choice(nodes1), random.choice(nodes2)
    n1.value, n2.value = n2.value, n1.value
    n1.children, n2.children = n2.children, n1.children
    return child1

def mutate(individual):
    mutant = copy.deepcopy(individual)
    nodes = get_all_nodes(mutant)
    target = random.choice(nodes)
    new_subtree = generate_random_tree(2)
    target.value = new_subtree.value
    target.children = new_subtree.children
    return mutant

def evolve(pop, generations=50):
    for gen in range(generations):
        pop.sort(key=fitness, reverse=True)
        next_gen = pop[:10] # Elitism: keep top 10

        while len(next_gen) < len(pop):
            if random.random() < 0.7:
                p1, p2 = random.sample(pop[:20], 2)
                next_gen.append(crossover(p1, p2))
            else:
                p = random.choice(pop[:20])
                next_gen.append(mutate(p))

        pop = next_gen
        best = max(pop, key=fitness)
        if gen % 10 == 0:
            print(f"Gen {gen}: Best Fitness = {fitness(best):.4f}")
    return max(pop, key=fitness)

final_best = evolve(population)
print(f"\nFinal Evolved Expression: {final_best}")
print(f"Final Fitness: {fitness(final_best):.4f}")

... Gen 0: Best Fitness = 0.0236
    Gen 10: Best Fitness = 0.5205
    Gen 20: Best Fitness = 0.9631
    Gen 30: Best Fitness = 1.0000
    Gen 40: Best Fitness = 1.0000

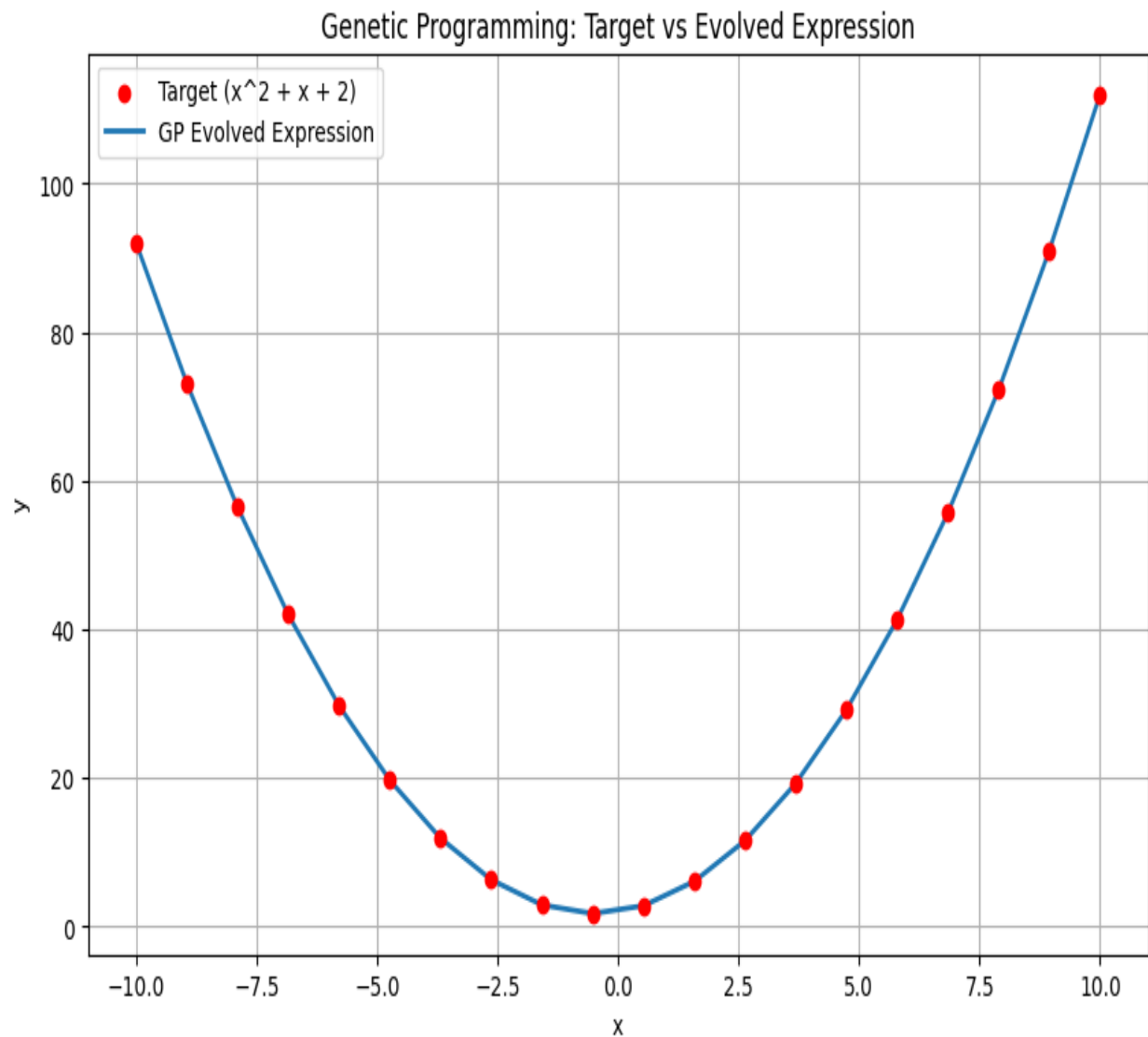
Final Evolved Expression: (add (sub (mul x x) (mul (sub (add x 1.041972977805222) (sub (sub -3.790003869
Final Fitness: 1.0000
```

```
import matplotlib.pyplot as plt

# Generate predictions from the best individual
y_pred = np.array([final_best.evaluate(x) for x in x_train])

plt.figure(figsize=(10, 6))
plt.scatter(x_train, y_train, color='red', label='Target ( $x^2 + x + 2$ )', zorder=5)
plt.plot(x_train, y_pred, label='GP Evolved Expression', linewidth=2)
plt.title('Genetic Programming: Target vs Evolved Expression')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True)
plt.show()

print(f"Best Formula Found: {final_best}")
```



[5]
✓ 3s

```
import copy

def get_all_nodes(node):
    nodes = [node]
    for child in node.children:
        nodes.extend(get_all_nodes(child))
    return nodes

def crossover(parent1, parent2):
    child1, child2 = copy.deepcopy(parent1), copy.deepcopy(parent2)
    nodes1, nodes2 = get_all_nodes(child1), get_all_nodes(child2)

    n1, n2 = random.choice(nodes1), random.choice(nodes2)
    n1.value, n2.value = n2.value, n1.value
    n1.children, n2.children = n2.children, n1.children
    return child1

def mutate(individual):
    mutant = copy.deepcopy(individual)
    nodes = get_all_nodes(mutant)
    target = random.choice(nodes)
    new_subtree = generate_random_tree(2)
    target.value = new_subtree.value
    target.children = new_subtree.children
    return mutant

def evolve(pop, generations=50):
    for gen in range(generations):
        pop.sort(key=fitness, reverse=True)
        next_gen = pop[:10] # Elitism: keep top 10

        while len(next_gen) < len(pop):
            if random.random() < 0.7:
                p1, p2 = random.sample(pop[:20], 2)
                next_gen.append(crossover(p1, p2))
            else:
```

[3]
✓ 3s

```
                p = random.choice(pop[:20])
                next_gen.append(mutate(p))
        pop = next_gen
        best = max(pop, key=fitness)
        if gen % 10 == 0:
            print(f"Gen {gen}: Best Fitness = {fitness(best):.4f}")
    return max(pop, key=fitness)

final_best = evolve(population)
print(f"\nFinal Evolved Expression: {final_best}")
print(f"Final Fitness: {fitness(final_best):.4f}")
```

▼

```
... Gen 0: Best Fitness = 0.0017
Gen 10: Best Fitness = 0.2005
Gen 20: Best Fitness = 0.7346
Gen 30: Best Fitness = 0.9243
Gen 40: Best Fitness = 0.9587
```

```
Final Evolved Expression: (sub (sub (mul x x) (mul (mul 4.774602652626731 (mul 4.801545479687379 (add 1.
Final Fitness: 0.9819
```